Mixture of experts in Machine Learning: Theoretical aspects and possible applications.

Graduand
Davide Avolio

Supervisor
Alessandro Mei



SAPIENZA
UNIVERSITÀ DI ROMA



SAPIENZA
UNIVERSITÀ DI ROMA

Mixture of experts in Machine Learning: Theoretical aspects and possible applications.

Faculty of Information Engineering, Computer Science and Statistics
Bachelor's Degree in Computer Science

Graduand
Davide Avolio
ID 2072670

Supervisor
Alessandro Mei

Academic year 2024-2025

Index

Introduction _____ *3*

1. Preprocessing _____ *4*

1.1. Dataset level 4
1.2. Sentence level 4

2. Tokenization _____ *4*

2.1. Byte-pair encoding 4
2.2. WordPiece 5
2.3. Unigram tokenization 5
2.4. SentencePiece 5

3. Word embedding _____ *5*

3.1. Traditional approach 6
3.2. Neural approach 6
3.3. Embeddings from LLMs 8

4. Positional encoding _____ *8*

4.1. Absolute positional encoding 9
4.2. Relative positional encoding 9
4.3. Rotary positional encoding 10

5. Activation functions _____ *10*

5.1. Sigmoid 11
5.2. SiLU 11
5.3. ReLU 11
5.4. Leaky ReLU 11
5.5. Exponential Linear Units 11
5.6. Softplus 12
5.7. Tanh 12
5.8. Gaussian Error Linear Unit 12
5.9. Gated Linear Unit 12
5.10. GeGLU 12
5.11. Swish 12
5.12. SwiGLU 12

6. Normalization _____ *12*

6.1. Batch Normalization 13
6.2. Layer normalization 13

7. Model components _____ *13*

7.1. Recurrent Neural Networks 13

7.2. LSTM 15
7.3. Attention 17
7.4. Transformer (Multi-Head) 18
7.5. Grouped-query attention 21
7.6. Multi-head Latent Attention 21

8. Macro-architectural modalities *22*
8.1. Bagging 22
8.2. Boosting 22
8.3. Stacking 23
8.4. Mixture of experts 23

9. Training *23*
9.1. Distillation 23
9.2. Reinforcement learning from human feedback 24
9.3. Reinforcement learning from AI feedback 24
9.4. Group Relative Policy Optimization 25
9.5. Parameter Efficient Fine-Tuning 25

10. Prompting *26*
10.1. Zero-shot prompting 26
10.2. Few-shot prompting 27
10.3. Chain of thoughts 27
10.4. Meta prompting 27
10.5. Generated Knowledge prompting 27
10.6. Tree of Thoughts 28

11. Mixture of experts *28*

12. Models *32*

13. DeepSeek R1 ..... *34*

14. Practical experimentation *34*
14.1. The data used 34
14.2. Preprocessing 36
14.3. Approaches to the problem 36
14.4. The model chosen 37
14.5. Training 38

15. Conclusions *42*

Introduction

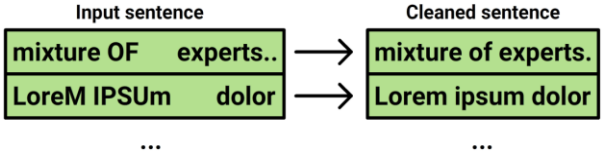
Nowadays a large number of models and their variants about artificial intelligence, machine learning and deep learning are being published every day. Given this, I desired to study a technique rather than a specific model.

Between the end of 2024 and the beginning of 2025, the release of a series of LLMs from DeepSeek company made headlines. I therefore decided to study in detail what were the real innovations of these models and I found out that one of the main characteristics was the use of the so-called Mixture of Experts (MoE) method integrated on top of the Transformer architecture. This method fits perfectly the idea of looking for a general technique.

I then decided to dedicate the internship activity to the study of this technique. It was convenient to divide it in three stages:

1. Study of common LLM components and training techniques: before analysing directly the MoE, it's mandatory to study the components and phases of an LLM so to have a general knowledge on the topic.
2. In-depth analysis on the MoE: the specific study of this technique and the main models using them.
3. Practical experimentation: testing a model on a practical task.

1. Preprocessing



1.1. Dataset level

When choosing our dataset, we can adopt several strategies to ensure its quality:

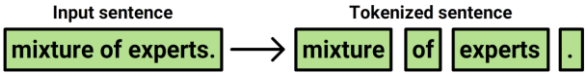
- Deduplication: Removing duplicate documents from the dataset. There're several techniques to detect the duplicates, for example we can use a hash function to each document and check the similarity of each hash.
- Filtering: filtering and evaluating data from a linguistic and semantic point of view, to ensure its quality.
- Remixing: representing more undercovered domains so to have a more balanced and inclusive dataset.

1.2. Sentence level

When analysing the sentences themselves instead, we can adopt the following techniques (only if they don't bring to a loss in the meaning):

- Converting to lowercase: otherwise, for example "Cat", "cat", "CAT" and "cAT" would be considered as different words.
- Removing non-words: such as URLs, punctuation marks, multiple whitespaces and symbols that don't provide any meaningful information.

2. Tokenization



2.1. Byte-pair encoding

Byte-pair encoding (BPE) is an algorithm used to encode a string of characters into a string of tokens. It can be done in two levels: at a character level or at a byte level. The character-level BPE operates in this way: we assign a token to all the characters that appear in the text and then to the most frequent pairs in the text (the algorithm is hence greedy). For example, the string "abacababac" can be encoded as "0320332" with $a = 0, b = 1, c = 2, ba = 3$ or as "0102010102" if our vocabulary has only 3 tokens. The limitations of this algorithm are the impossibility of performing the encoding in case of unseen characters and the elevate number of tokens to use to encode the Unicode characters (Unicode 16.0 has 155,063 characters). These problems are solved by the byte-level BPE (BBPE), that, instead of encoding directly the characters, it converts them into a stream of bytes that will then be encoded, allowing a

representation of all the Unicode characters with a limited number of tokens. Here we have some tokens for the frequent pairs too.

When we have to actually encode a sentence into tokens, we will split each character in Unicode (in case of BPE) or bytes (in case of BBPE) and apply the merge rules.

(Gage, 1994)

(Wang, Cho, & Jiatao, 2019)

2.2. WordPiece

As in BPE, WordPiece starts from a vocabulary including the special tokens used by the model and the initial alphabet. Since it identifies subwords by adding a prefix (like ## for BERT), each word is initially split by adding that prefix to all the characters inside the word (word is "w ##o ##r ##d"). The vocabulary is so made of all the characters that appear at the beginning of the words and all the characters that appear inside a word preceded by the WordPiece prefix. Here we have merge rules too, but in this case, for each pair, the algorithm will merge the two parts only if the individual parts are less frequent in the vocabulary; for example, in ("un", "##able") we might not merge the two parts because "##able" could be more frequent than "unable", while in ("c", "##at"), "cat" could be more frequent, so we merge the two.

When we have to actually encode a sentence into tokens, we will search for the longest subword and proceed iteratively with the rules.

(Wu, Schuster, & Zhifeng, 2016)

2.3. Unigram tokenization

In the unigram tokenization the vocabulary is made of the base vocabulary and of the most common pairs that appear; for example if we have the words "cat" and "bat", the dictionary will be made of {a, b, c, t, ca, at, ba, cat, bat}. At every step one pair is removed and the tokenization loss is computed; the pair that makes the loss increase the less is removed and these steps are repeated until the wanted size of the vocabulary is reached.

When we have to actually encode a sentence into tokens, we will search for the most probable division based on the scores learned during the tokenization phase.

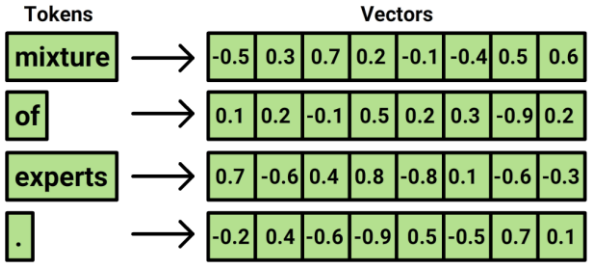
(Kudo, 2018)

2.4. SentencePiece

SentencePiece is a library of tokenizer algorithms that contains optimized BPE, WordPiece and Unigram.

(Kudo & Richardson, A simple and language independent subword tokenizer and detokenizer for Neural Text Processing, 2018)

3. Word embedding



Word embedding is a numerical representation of words in a low-dimensional space that can be later used as input for machine learning models. There're two approaches: in the traditional one, a measure of frequency is evaluated and the vector doesn't contain the meaning of that word, while in the neural approach, a neural network is used and the vectors capture the relationship between words and their meaning.

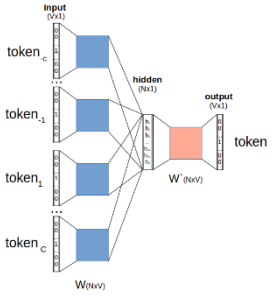
3.1. Traditional approach

- One-Hot encoding: we enumerate all the words in a dictionary and assign to each word a vector, whose dimensionality is the same one as the vocabulary and it has all zeros but the element corresponding to the index of the word in the vocabulary, which is set to 1.
- TF-IDF: It's composed of two parts. The Term Frequency (TF) measures how frequent a word is, so it's $F\ t,d\ = \frac{Total\ number\ of\ times\ word\ t\ appears\ in\ d}{Total\ number\ of\ words\ in\ a\ document\ d}$; the Inverse Document Frequency (IDF) measures how rare a word is (so it might be significant for that document) and it's $IDF\ t,D\ = \log\left(\frac{Total\ documents}{Number\ of\ documents\ containing\ t}\right)$. Finally, the TF-IDF value is evaluated as $TF\ t,d\ \cdot IDF\ t,D$; the higher is the score and the more important that term is to that document.
- Bag-of-words (BOW): each document is an unordered set of words and each entry of the vector is the respective frequency.

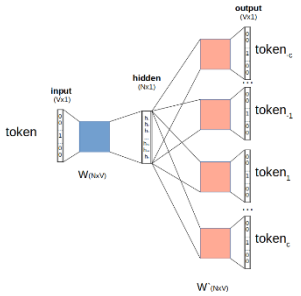
3.2. Neural approach

A neural approach for generating word embeddings is Word2Vec; there're two methods for Word2Vec:

- Continuous bag of words (CBOW): it's a feedforward network that has a single hidden layer. In the input layer there're the context words, and the output layer is the target word. The weights between the input and hidden layer are learned during training.



- Skip-gram: it's a feedforward network where the input layer is the target word and the output layer are the context words.



Other approaches are

- Global Vectors (GloVe): this approach doesn't use a context window, but a co-occurrence matrix. A co-occurrence matrix is a matrix X that has in both rows i and columns j words and the value X_{ij} is a counter of how many times the word in a column, appeared beside the word in a row. $X_i = \sum_k X_{ik}$ is the number of times any word appears in the context of word i. $\mathbb{P}_{ij} = \mathbb{P}\ j|i\ = \frac{X_{ij}}{X_i}$ is the probability that word j appears in the context of word i. With $\frac{\mathbb{P}_{ik}}{\mathbb{P}_{jk}}$ and a fixed word k, we'll get a high ratio if k is related to i but not j, a low ratio if k is related to j but not i and a ratio close to 1 if k is either unrelated or related to both. We can use a function F to express this:

$$F(w_i, w_j, w_k) = \frac{\mathbb{P}_{ik}}{\mathbb{P}_{jk}}$$

Where w and w both are in $\mathbb{R}^d$ and w is a separate context word vector. In order to pick a specific function, we'll simplify it with some rules. Since we're interested in encoding the information present in the ratio and since we're in a vector space, we consider the difference of w_i and w_j; moreover, the result of the ratio is a scalar, so we use the dot product to obtain a scalar.

$$F((w_i - w_j)^T w_k) = \frac{\mathbb{P}_{ik}}{\mathbb{P}_{jk}}$$

Since the distinction between a word and a context word is arbitrary, we need $w \longleftrightarrow w$ and $X \leftrightarrow X^T$. We can achieve this with a homomorphism between the groups $\mathbb{R}, +$ and $\mathbb{R}_{>0}, \times$:

$$F((w_i - w_j)^T w_k) = \frac{F\ w_i^T w_k}{F(w_j^T w_k)}$$

And from the previous equation we get that:

$$F\ w_i^T w_k\ = \mathbb{P}_{ik} = \frac{X_{ik}}{X_i}$$

The function that has this property is the exponential, so we get that:

$$w_i^T w_k = \log\ \mathbb{P}_{ik}\ = \log\ X_{ik}\ - \log\ X_i$$

To ensure the symmetry and since $\log\ X_i$ doesn't depend on k, we add two biases:

$$w_i^T w_k + b_i + b_k = \log\ X_{ik}$$

Since the logarithm diverges at 0 and since it's weighting all the co-occurrences equally, even the rare ones, we define the loss function as:

$$J = \sum_{i,j=1}^{V} f(X_{ij})\left(w_i^T w + b_i + b_j - \log(X_{ij})\right)^2$$

Where V is the size of the vocabulary, $f\ 0\ = 0$, $f\ x$ it's non decreasing so that rare co-occurrences are not overweighted and it should be relatively small for large values of x so to not overweight frequent co-occurrences. A function that satisfies these properties is:

$$f\ x\ = \begin{cases} \left(\frac{x}{x_{max}}\right)^{\alpha}, & if\ x < x_{max} \\ 1, & otherwise \end{cases}$$

A value of x_{max} is 100 and for α is 0.75 ($\alpha = 1$ is linear). For the values of X_{ij} that are 0, the cost function is computed immediately, avoiding unrequited computation.

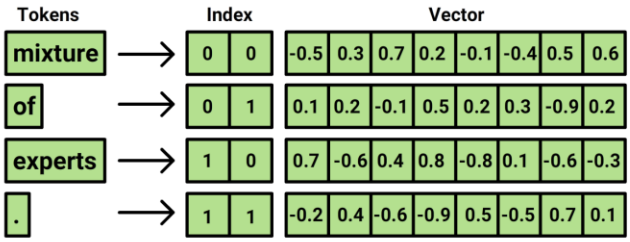
3.3. Embeddings from LLMs

- Embedding from Language Models (ELMo): there're two LSTM networks that process the input sequence in opposite directions (forward and backward) and their output is concatenated to form a single vector representation for each word in the input sequence.
- Bidirectional Encoder Representations from Transformers (BERT): In BERT we have a bidirectional encoding too, but the difference with the other techniques is the use of multiple transformer layers that can capture more complex contextual information; moreover, the order is significant. Another feature of BERT is the use of sub-tokens, that can allow words to be split in smaller parts; this can be useful for example in case of Out Of Vocabulary (OOV) words.

(Peters, Neumann, & Iyyer, 2018)

(Devlin, Chang, & Lee, 2018)

4. Positional encoding

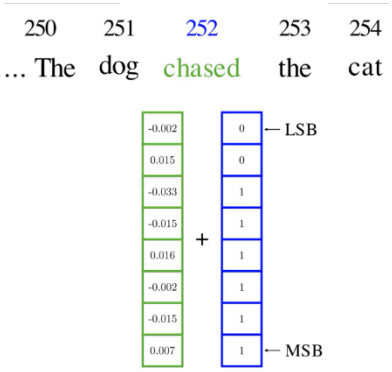


When we obtain the embedding of a token, especially in architectures with the Self-Attention module, the order is not represented; for example, Self-Attention will consider the sentences "The dog chased the cat" and "The cat chased the dog" the same. To solve this problem, we add to each word vector another vector that encodes the position. This function should have four properties:

- It should output a unique encoding for each position.
- The distance between any two positions should be consistent across sentences with different lengths.
- The model should generalize to longer sentences and be bounded.

- It must be deterministic (it shouldn't change accordingly to the batch in which we are).
- It should be extensible to multiple dimensions.

4.1. Absolute positional encoding



In the absolute positional encoding, we use the absolute position of the tokens; however, adding the absolute position doesn't work because it would make the values of the word embedding negligible relative to the total. Normalizing those values dividing them by the length of the sentence would make the values of the PE dependent from the length of the sentence.

Instead of adding values that are greater than 1, the ideal would be staying in a range [0,1], so we still keep the position, but we express it with binary numbers and add to each dimension of the vector a digit of the position. Since the least significant bit (LSB) cycles at every token, while the MSB will cycle every 2^{n-1} tokens (n is the number of bits), we can express this with a sinusoidal encoding. Given the position pos or k and given the dimension of the vector d, we need to:

$$PE_{(k,2i)} = sin(\frac{pos}{10000^{\frac{2i}{d_{model}}}})$$

$$PE_{(k,2i+1)} = cos(\frac{pos}{10000^{\frac{2i}{d_{model}}}})$$

$$k \begin{cases} P(0) = \left[sin(\frac{0}{10000^{\frac{0}{4}}}), cos(\frac{0}{10000^{\frac{0}{4}}}), sin(\frac{0}{10000^{\frac{1}{4}}}), cos(\frac{0}{10000^{\frac{1}{4}}})\right] \\ P(1) = \left[sin(\frac{1}{10000^{\frac{0}{4}}}), cos(\frac{1}{10000^{\frac{0}{4}}}), sin(\frac{1}{10000^{\frac{1}{4}}}), cos(\frac{1}{10000^{\frac{1}{4}}})\right] \\ P(2) = \left[sin(\frac{2}{10000^{\frac{0}{4}}}), cos(\frac{2}{10000^{\frac{0}{4}}}), sin(\frac{2}{10000^{\frac{1}{4}}}), cos(\frac{2}{10000^{\frac{1}{4}}})\right] \\ P(3) = \left[sin(\frac{3}{10000^{\frac{0}{4}}}), cos(\frac{3}{10000^{\frac{0}{4}}}), sin(\frac{3}{10000^{\frac{1}{4}}}), cos(\frac{3}{10000^{\frac{1}{4}}})\right] \\ P(4) = \left[sin(\frac{4}{10000^{\frac{0}{4}}}), cos(\frac{4}{10000^{\frac{0}{4}}}), sin(\frac{4}{10000^{\frac{1}{4}}}), cos(\frac{4}{10000^{\frac{1}{4}}})\right] \\ P(5) = \left[sin(\frac{5}{10000^{\frac{0}{4}}}), cos(\frac{5}{10000^{\frac{0}{4}}}), sin(\frac{5}{10000^{\frac{1}{4}}}), cos(\frac{5}{10000^{\frac{1}{4}}})\right] \end{cases}$$

$d_model = 4$

i=0 i=0 i=1 i=1

We can also define an angular velocity: $\omega_k = \frac{1}{10000^{2k/d}}$. 10000 is the base wavelength, so every 10000 we're resetting our count (position 0 and 10000 are the same), but in this case we won't suffer of the cyclic problem because we won't have that long sentences.

The problem with the absolute positional encoding is that during training, the model might give different meanings to the same sentences when they're in different unimportant arrangements.

4.2. Relative positional encoding

In the relative positional encoding, we maintain the relative position of words independently of the sentence shift. In this case we're not adding to the word a position vector, but we're adding every time a bias to the previous and following vectors that depends on the distance with respect to the current vector. The limitations of Relative Positional encodings are that they're slow with long sequences and they alter the embedding itself.

(Shaw, Uszkoreit, & Vaswani, 2018)

4.3. Rotary positional encoding

The rotary positional encoding (RoPE) mixes both the absolute and the relative ones; adding a positional vector to our words, modifies its embedding and its norm. When using the self-attention mechanism, we want that when we look up a word (query) in our dictionary (keys), nearby words should have more influence than distant ones. This influence is determined by the dot product:

$$q \cdot k = |q|\ k\ \cos\theta$$

Where θ is the misalignment in radians between the two vectors. In order to modify the value of the dot product without changing the norm, we have to rotate our vectors. Before doing so, we'll consider each vector as a merge of $d/2$ pairs of 2-D vectors. In this way, we encode the relative position by multiplying each pair with a rotation matrix so defined; for an input vector q at position p, we have:

$$R\ q,p\ = \begin{pmatrix} M_1 & 0 & 0 & 0 \\ 0 & M_2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & M_{d/2} \end{pmatrix} \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_d \end{pmatrix}$$

$$M_i = \begin{bmatrix} \cos\ \omega_i p & \sin\ \omega_i p \\ -\sin\ \omega_i p & \cos\ \omega_i p \end{bmatrix}$$

Since we have a sparce matrix, we can directly compute:

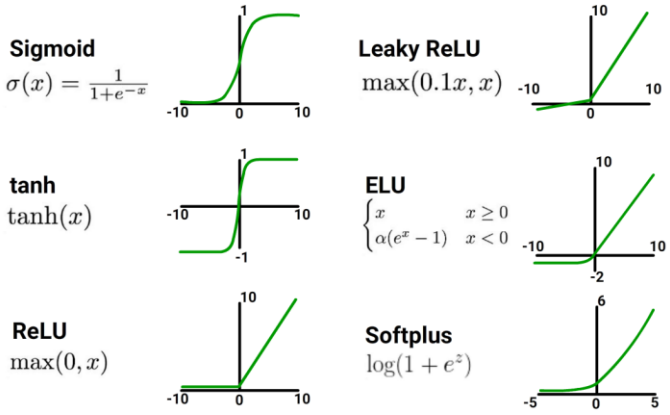
$$R^d_{\Theta,p} = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{d-1} \\ q_d \end{pmatrix} \otimes \begin{pmatrix} \cos\ p\theta_1 \\ \cos\ p\theta_1 \\ \cos\ p\theta_2 \\ \cos\ p\theta_2 \\ \vdots \\ \cos(p\theta_{d/2}) \\ \cos(p\theta_{d/2}) \end{pmatrix} + \begin{pmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_d \\ q_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin\ p\theta_1 \\ \sin\ p\theta_1 \\ \sin\ p\theta_2 \\ \sin\ p\theta_2 \\ \vdots \\ \sin(p\theta_{d/2}) \\ \sin(p\theta_{d/2}) \end{pmatrix}$$

The values of θ will be the same in all the words at the same position. The ω_i is the same as the one in the absolute positional encoding.

(Vaswani, Shazeer, & Parmar, 2017)

(Su, Lu, & Pan, 2021)

5. Activation functions



5.1. Sigmoid

The sigmoid can model output's probability:

$$\sigma\ z\ = \frac{1}{1+e^{-z}}$$

Sigmoid goes from 0 to 1 but has the problem of killing the gradient: for too high or low values of z, the Sigmoid will give as output 0. Another problem is that the local gradient of the sigmoid is always positive, so the sign of the final loss only depends on the upstream gradient (so either is all positive or all negative).

5.2. SiLU

The Sigmoid Linear Unit (SiLU) is an activation function derived from the sigmoid:

$$SiLU\ z\ = z\sigma\ z$$

5.3. ReLU

ReLU (Rectified Linear Unit) is defined as:

$$ReLU\ z\ = \max\ 0, z$$

The ReLU instead doesn't saturate in the positive region, is computationally efficient (no exponentials), it doesn't have negative values, converges much faster than sigmoid (around 6 times) but with negative values, will never activate, so never updates and it's not differentiable in zero (so usually we assume that the derivative in 0 is 0).

5.4. Leaky ReLU

Leaky ReLU is defined as:

$$LeakyReLU\ z\ = \max\ 0.01z, z$$

Leaky ReLU fixes the problem of not being differentiable. In the Parametric Rectifier (PreLU) instead of having 0.01 we have a parameter α.

5.5. Exponential Linear Units

Exponential Linear Units (ELU) is defined as:

$$ELU\ z\ = \begin{cases} z, & if\ z > 0 \\ \alpha\ e^z - 1\ , & if < z \leq 0 \end{cases}$$

The ELU solves the problem of not being differentiable, but it requires exponentials in the computation.

5.6. Softplus

Softplus is a smooth ReLU:

$$softplus\ z\ = \log\ 1 + e^z$$

5.7. Tanh

Tanh is a scaled and shifted version of the sigmoid, that goes from -1 to 1:

$$\tanh\ z\ = \frac{e^{2z} - 1}{e^{2z} + 1}$$

5.8. Gaussian Error Linear Unit

Gaussian Error Linear Unit (GELU) is defined as:

$$GELU\ z\ = z\Phi\ z$$

Where $\Phi\ z$ is the standard Gaussian cumulative distribution function.

5.9. Gated Linear Unit

The Gated Linear Unit (GLU) is so defined:

$$GLU\ X\ =\ X \cdot W + b\ \otimes \sigma\ X \cdot V + c$$

It's inspired to LSTMs and it uses the sigmoid function σ, among with two set of weights W and V and two biases b and c. Where $\otimes$ is an element-wise product.

5.10.GeGLU

GeGLU is the union of GELU and GLU:

$$GeGLU\ x, W, V, b, c\ = GELU\ xW + b\ \otimes\ xV + c$$

5.11.Swish

The Swish function is defined as:

$$Swish_\beta\ x\ = x\sigma\ \beta x$$

Where σ is the sigmoid and β is a learnable parameter.

5.12.SwiGLU

The SwiGLU activation function is the union of the Swish one and the GLU:

$$SwiGLU\ x, W, V, b, c, \beta\ = Swish_\beta\ xW + b\ \otimes\ xV + c$$

(Shazeer, GLU Variants Improve Transformer, 2020)

6. Normalization

6.1. Batch Normalization

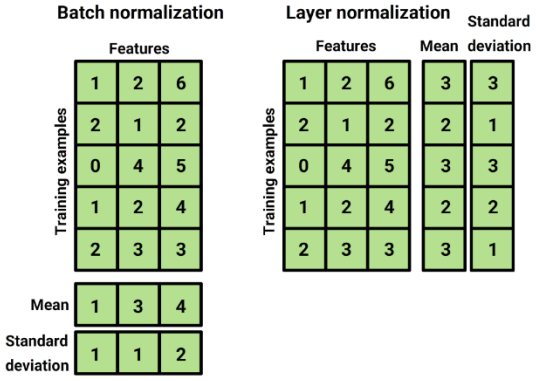
In batch normalization we normalize along the training samples. This is used more frequently in computer vision contexts.

$$\frac{y_{n,j}-\mu_j}{\sigma_j}, \qquad \mu_j=\frac{1}{N}\sum_{n=1}^{N}x_{n,j}\ , \qquad \sigma_j^2=\frac{1}{N}\sum_{n=1}^{N}\left(x_{n,j}-\mu_j\right)^2$$

Where N is the number of samples.

6.2. Layer normalization

In layer normalization we compute the average along the features, so we're able to learn from data with minibatch sizes; moreover, since there're no dependencies between data, we can do parallel computations, useful whenever we have large text corpora. Layer normalization is preferred for NLP tasks because the sentence length may vary.



$$\frac{y_{i,d}-\mu_i}{\sigma_i}, \qquad \mu_i=\frac{1}{D}\sum_{d=1}^{D}x_{i,d}\ , \qquad \sigma_i^2=\frac{1}{D}\sum_{d=1}^{D}(x_{i,d}-\mu_i)^2$$

Where D is the dimension of each vector.

7. Model components

7.1. Recurrent Neural Networks

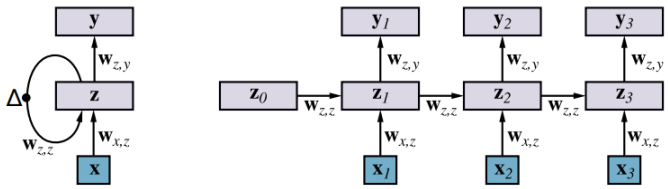
Architecture

Recurrent neural networks (RNNs) can accept cycles in their computational graph. Every cycle has a delay, so that the RNN can have an internal state or memory: inputs received at earlier time steps affect the RNN's response to the current input. The hidden layer z and the output layer y are so defined:

$$z_t=f_w\ z_{t-1},x_t\ =g_z(W_{z,z}z_{t-1}+W_{x,z}x_t)\equiv g_z(in_{z,t})$$
$$y_t=g_y(W_{z,y}z_t)\equiv g_y(in_{y,t})$$

Where g_z and g_y denote the activation functions for the hidden and output layers, respectively. So, in z we are balancing the previous output with the input, while in y we are only using an activation function.

We can unroll this model converting it into a feedforward network, but we have the shared weights $W_{x,z}$, $W_{z,z}$ and $W_{z,y}$.



Assuming to have an RNN with one input unit, one hidden unit and one output unit, with the explicit bias, the formulas will be:

$$z_t = g_z(w_{z,z}z_{t-1} + w_{x,z}x_t + w_{0,z})$$
$$y_t = g_y(w_{z,y}z_t + w_{0,y})$$

For the loss function we use the squared error between the given output and the truth. In order to tune $w_{z,z}$ we have to compute $\frac{\partial \mathcal{L}}{\partial w_{z.z}}$:

$$\frac{\partial \mathcal{L}}{\partial w_{z.z}} = \frac{\partial}{\partial w_{z.z}} \sum_{t=1}^{T} \ y_t - y_t \ ^2 = \sum_{t=1}^{T} -2 \ y_t - y_t \ \frac{\partial y_t}{\partial w_{z.z}}$$

Now we replace the last y_t with its definition:

$$= \sum_{t=1}^{T} -2 \ y_t - y_t \ \frac{\partial}{\partial w_{z.z}} g_y(in_{y,t}) = \sum_{t=1}^{T} -2 \ y_t - y_t \ \ g'_y(in_{y,t}) \frac{\partial}{\partial w_{z.z}} in_{y,t}$$

$$= \sum_{t=1}^{T} -2 \ y_t - y_t \ \ g'_y(in_{y,t}) \frac{\partial}{\partial w_{z.z}} (w_{z,y}z_t + w_{0,y})$$

$$= \sum_{t=1}^{T} -2 \ y_t - y_t \ \ g'_y(in_{y,t}) w_{z,y} \frac{\partial z_t}{\partial w_{z.z}}$$

To compute $\frac{\partial z_t}{\partial w_{z.z}}$ we have to use the first equation:

$$\frac{\partial z_t}{\partial w_{z.z}} = \frac{\partial}{\partial w_{z.z}} g_z(in_{z,t}) = g'_z(in_{z,t}) \frac{\partial}{\partial w_{z.z}} in_{z,t} = g'_z(in_{z,t}) \frac{\partial}{\partial w_{z.z}} (w_{z,z}z_{t-1} + w_{x,z}x_t + w_{0,z})$$

And by the rule for derivatives of products (since both $w_{z,z}$ and z_{t-1} depend on $w_{z,z}$):

$$= g'_z(in_{z,t}) \left(z_{t-1} + w_{z,z} \frac{\partial z_{t-1}}{\partial w_{z.z}} \right)$$

So, this equation is recursive, because it needs the value of z from step $t-1$.The total run time for computing the gradient will be linear in the size of the network (backpropagation through time). If we iterate the recursive calculation, the gradients at T will include terms proportional to $w_{z,z} \prod_{t=1}^{T} g'_z(in_{z,t})$; since in sigmoids, ReLUs and tanhs $g' \leq 1$, if $w_{z,z} < 1$, then we have the problem of vanishing gradients, while if $w_{z,z} > 1$ we have the opposite problem of the exploding gradient.

Even the RNN can be a one-to-many (one input and at every step we get an output), many-to-many or many-to-one model (from a sentence or a word we get a single output). For the loss we should run forward and backward pass through the entire sequence to compute the gradient, but we can also perform the truncated backpropagation and run both phases in chunks of the sequence. We can use RNNs to search interpretable cells such as quotes, if statements, or to keep track of line length or code depth.

Advantages and Limitations

The advantages of RNN are:

- They can process the context of the word.
- The number of weights of RNN is not high.
- They can process sequences of variable length

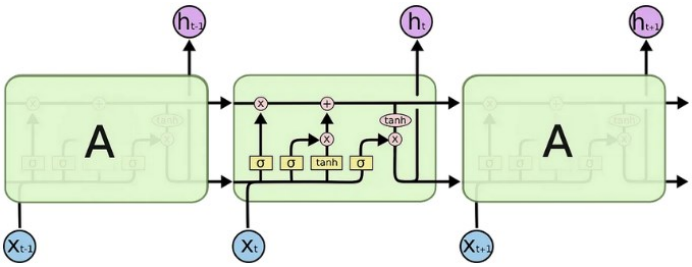
The limitations of RNN are:

- Vanishing or exploding gradient problem
- Since the memory is limited, the more the words and the more the initial data are corrupted.
- Since we're using recurrent components and we're processing data sequentially, RNN can't be parallelized.
- If we want to find the distance between an input and the corresponding output, we might have to travel all the architecture.

7.2. LSTM

Architecture

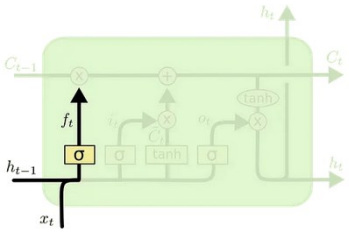
Based on the RNN, the Long Short-Term Memory (LSTM) model aims to solve the problem of information corruption. The idea behind this model is that by remembering or forgetting specific words, we can learn long-distance dependencies.



At the base of LSTM there's the LSTM node, made of 5 parts: the cell state that is passed down to every node in the chain, its update and other 3 gates (forget, input and output).

Apart from the operations, there're two types of neural network layers: the sigmoid (that returns values between 0 and 1) and the tanh (that returns values between -1 and 1).

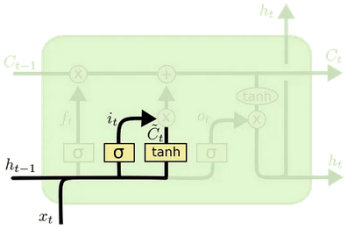
In the first gate, the forget gate, we determine if each element of the memory cell is remembered (and copied to the next time step) or forgotten (and set to 0). Here we combine the previous hidden layer and the input and we pass this through the first sigmoid. The sigmoid returns a vector made of values between 0 and 1; the more a value is close to 0 and the more we'll forget about it.



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Forget gate layer

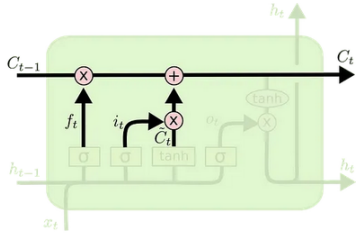
The combination of hidden layer and input now passes through the sigmoid layer again, to decide which values should be modified (the ones close to 1) and which ones should be kept unmodified (the ones close to 0) and, to decide how much we should modify those values, we use the tanh. This is the input gate:



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Sigmoid and Tanh deciding what to manipulate and by how much at input gate layer

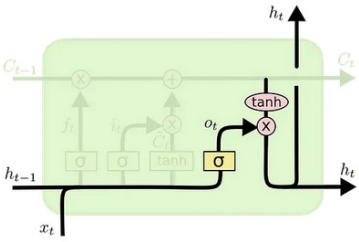
Now we take the previous cell state C_{t-1} forget the values given by f_i and modify the values given by $i_t * C_i$ (this is an element-wise product).



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Carrying out the calculations to actually update cell state

Finally, now that we have updated the cell, we use it to actually modify the hidden layer (output gate). The combination between h_{t-1} and x_t is passed through a sigmoid to see which values should be changed, while the cell is passed through a tanh, to see how much should we change the hidden layer.



$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Output gate calculations

Advantages and Limitations

The advantages of LSTMs are:

- It can handle inputs of various size.

- It can handle long sequences.
- It can remember important information and discard the irrelevant ones.

The limitations of LSTMs are:

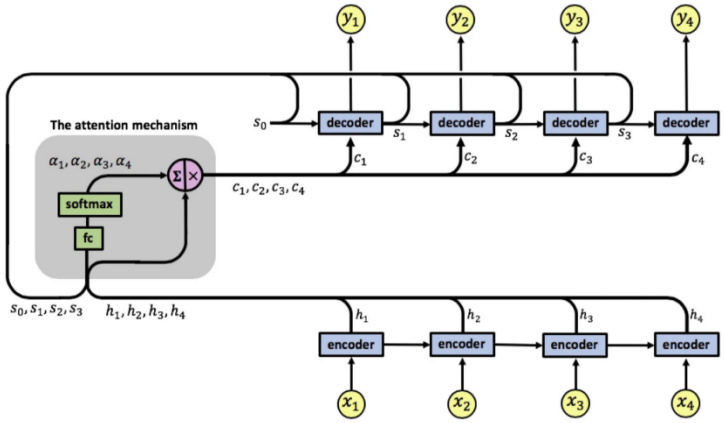
- It has a limited interpretability.
- It doesn't fully solve the problem of vanishing/exploding gradients.
- It's computationally expensive to train LSTMs.

(Hochreiter & Schmidhuber, 1997)

7.3. Attention

Architecture

In the Attention we have a sequence-to-sequence RNN with an additional component. After having encoded all the sequence, we take the last output of the decoder and all the results of the encoder and pass them to the attention mechanism to obtain a context vector c_i that will contain the most relevant information for generating the next target; the context vector will be used as an additional input to the target RNN.



The attention mechanism is divided in two steps: first we take all the outputs of the encoder and the last output of the decoder. At the time step t, we'll take the output s_{t-1} and all the previous outputs h_j (at the beginning s_0 is an empty vector); these are the alignment scores:

$$r_{tj} = s_{t-1} h_j$$

Then we do the SoftMax to get the attention weights:

$$a_{tj} = \frac{e^{r_{tj}}}{\sum_k e^{r_{tj}}}$$



So now, at each time step t, we have an attention coefficient for every hidden state output from the encoder. The attention is just the probability that each word will contribute to that output, so, now that we have these values, we do a linear combination (at each time step t) of the hidden states and we obtain the context vector c_t:

$$c_t = \sum_{j=1}^{T} a_{tj} h_j$$

The output of the decoder will be finally given by the concatenation of the input and the context vector.

Advantages and Limitations

The advantages of the attention mechanism are:

- Using different context vectors allows to reduce the bottleneck through a single vector.
- Thanks to the SoftMax, the attention coefficient are probabilities that make the attention differentiable (useful with back-propagation).
- The probabilities can solve the uncertainty because can distribute the probabilities over several options (and then the RNN will choose one).
- It can capture certain types of long-distance contextualization.
- Attention probabilities are often interpretable by humans.

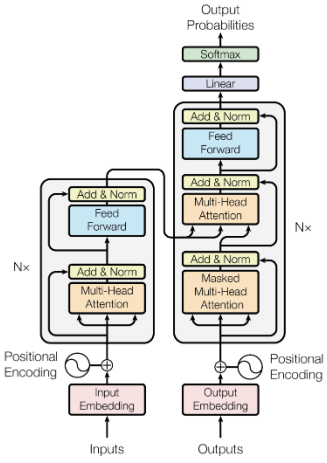
The limitations of the attention mechanism are:

- It's expensive to train
- It's a sequence-to-sequence model so the encoder and decoder are not parallelizable.

7.4. Transformer (Multi-Head)

Architecture

The transformer architecture is divided in encoder and decoder like a sequence-to-sequence model. The encoder is a stack of six identical layers (this number is just a choice made in the original paper), each composed by a multi-head self-attention and a feed forward layer (an MLP, so a fully connected layer). Normalization is done with layer norm, so that we operate with each vector individually, allowing parallelization. The decoder has the masked multi-head self-attention that receives the output embedding and then passes the result to a multi-head self-attention and then finally to a feed forward layer that will generate the output word.



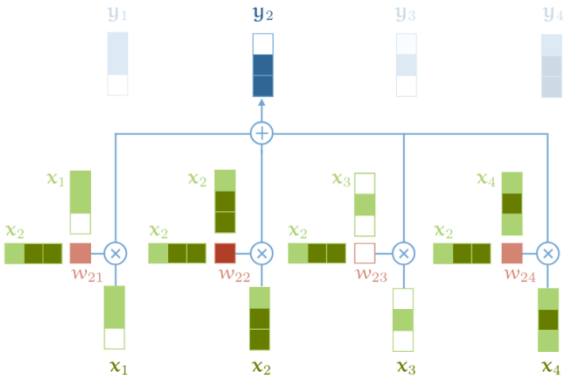
The main component of the transformer is the self-attention. While in the sequence-to-sequence models, attention was applied from the target RNN to the source RNN, in the self-attention, this mechanism is extended so that each sequence of hidden states also attends itself. Given a sequence of input vectors $x_1, x_2, \dots, x_t$, we obtain another sequence of vectors $y_1, y_2, \dots, y_t$, where each vector y_i is a weighted average of all the input vectors:

$$y_i = \sum_j a_{ij} x_j$$

And every weight is given by:

$$w_{ij} = x_i^T x_j, \qquad a_{ij} = \frac{e^{a_{ij}}}{\sum_j e^{w_{ij}}}$$

So, the weight is given by a function between x_i and x_j, the simples of which is the dot product between the two; this function evaluates the similarity between these vectors (obviously the similarity between a vector and itself will be really high). To ensure that the values are between 0 and 1 and they sum up to 1, we apply soft max to a_{ij}.



If we want to perform self-attention on all the vectors, then we need to represent the vectors as a $t \times k$ matrix (X) and compute XX^T. We then apply a row-wise soft max and multiply these weights with X.

In order to make self-attention learnable, we add some weights. Indeed, every vector x_i is used in three different ways:

- In $\sum_i a_{ki} x_i$, inside the weight a_{ki}, it's compared to every other vector to find its own weight (query).
- In $\sum_j a_{ij} x_j$, inside the weight a_{ij}, it's compared to every other vector to find the weight of the vector x_j (key).
- In $\sum_i a_{ki} x_i$, to find the weighted sum (value).

We add three $k \times k$ weight matrices W_q, W_k, W_v to find a linear transformation of x_i:

$$q_i = W_q x_i, \qquad k_i = W_k x_i, \qquad v_i = W_v x_i$$

$$w_{ij} = q_i^T k_i$$

$$a_{ij} = softmax(w_{ij})$$

$$y_i = \sum_j a_{ij} v_j$$

Since the SoftMax function is sensitive to very large input values (that may let the gradient drop to 0 and slow down the learning) and since the average value of the dot product grows with the dimension d, then, to improve it, we can scale the dot product by a factor of $\sqrt{d}$. When we have a vector $v \in \mathbb{R}^d$ whose value in every dimension is c, then its Euclidean length

will be $\sqrt{c^2 + \cdots + c^2} = \sqrt{dc^2} = c\sqrt{d}$, so $\sqrt{d}$ is how much is the length increasing with the dimensions. In this case then:

$$a_{ij} = softmax\left(\frac{w_{ij}}{\sqrt{d}}\right)$$

Self-attention takes as input a set, so the order of the elements doesn't matter (is permutation equivariant); when we swap two inputs, the corresponding outputs will be swapped too and this is why we use positional encoding before the self-attention.

In the masked self-attention layer, we prevent vectors from looking at future vectors; we manually set alignment scores to $-\infty$ so that we can parallelize attention across time; we also don't need to calculate the context vectors from the previous timestep. This is usually done if we want to avoid getting information of the future tokens during training.

Once we have a transformer model, if we want to train it to a specifical task, we first do a pretraining step, that consist in feeding the model with $i-1$ ground-truth words and then ask the model what's the ith word. Once this is done, there are two approaches, the feature-based approach and the fine-tuning approach. In the first one we mask some of the inputs and (by using both ends) we ask the model to fill the blanks, so that it can capture more information about a sentence; we usually mask 15% of the input. This is performed to encourage with bidirectional learning (LMs are indeed unidirectional, but language understanding is bidirectional). In the fine-tuning approach, we update the pre-trained model parameters via backpropagation; in this case we add another fully connected layer to the pre-trained one to accomplish certain tasks such as classification and then update the whole model based on the prediction performance on the labelled training set.

In order to capture information from different parts of the input, we use the multi-head self-attention (MHA). The three matrices W_q, W_k and W_v are gathered together in one set called head and then we take h of this heads; since they're independent, the computations can be done in parallel. After we get h of these vectors, we concatenate them in a $\mathbb{R}^{d \cdot h}$ vector and then use a linear projection to map it back to $\mathbb{R}^d$. This result is then passed in the linear combination.

Advantages and Limitations

The advantages of the transformer architecture are:

- They can be easily parallelized.
- They can capture long-term dependencies.
- They can work with any kind of sequential data.
- If we want to find the distance between an input and the corresponding output, the distance is always the same.

The limitations of the transformer architecture are:

- They have high computational demand.
- The predictions it makes are hard to interpret.

(Vaswani, Shazeer, & Parmar, 2017)

7.5. Grouped-query attention

Architecture

In the classical transformer architecture, for each query we have an associated value and key. In the Multi-Query attention, we group all the queries with one single value and key, but this could lead to data deterioration while decoding. The Grouped-Query Attention (GQA) is in between the multi-head one and the multi-query: we divide the queries in groups and we assign to each group a single value and key. There're many ways to divide the queries in groups: we could fix a number n and select groups of n queries or we could use a metric, evaluate the similarity between queries and use the same pair of key-value to each group.

Advantages and Limitations

The advantages of GQA are:

- GQA is faster than the MHA.
- GQA requires less memory than MHA.

The limitations of GQA are:

- GQA's quality is reduced with respect to the one of MHA.
- The performance of GQA depends on a new hyperparameter: the groups.

(Ainslie, Lee-Thorp, & de Jong, 2023)

7.6. Multi-head Latent Attention

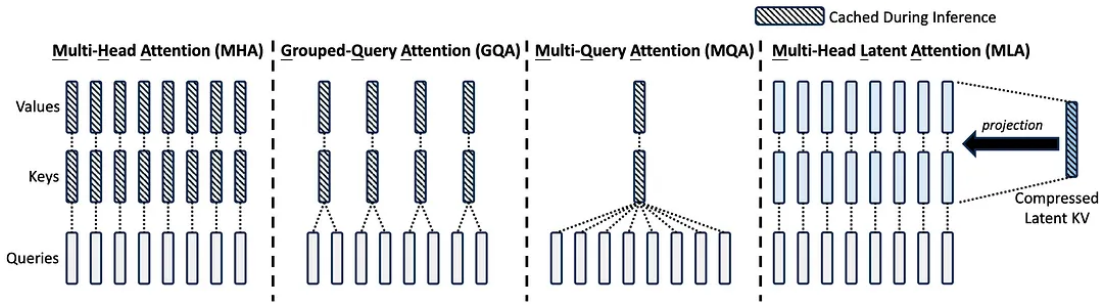
Architecture

In the Multi-Head Latent Attention (MLA), we compress all the keys and values and save only this compressed result. In the decoding phase, we'll bring it back to its original dimension. We have an input sequence $X \in \mathbb{R}^{T \times D}$, where T is the sequence length and D is the dimension and n_h heads. We use two matrices to compress the keys and the values, $W_K^a, W_K^b \in \mathbb{R}^{D \times r}$ and other two to decompress them, $W_K^b, W_V^b \in \mathbb{R}^{r \times \ n_h \cdot d_h}$. The new keys and values will be:

$$K = XW_K^a W_K^b \in \mathbb{R}^{T \times \ n_h \cdot d_h}$$
$$V = XW_V^a W_V^b \in \mathbb{R}^{T \times \ n_h \cdot d_h}$$

We can lower the rank (Low Rank Adaptation or LoRA) of key-value pairs or of the queries.

When using MLA with RoPE however, we need to modify this positional encoding method, because we need to save the relative position in separate vectors that are then concatenated with the compressed keys and values before the up-projection.



Advantages and Limitations

The advantages of MLA are:

- It lowers the memory usage
- It has a faster inference

The limitations of MLA are:

- The quality is reduced

(Meng, Yao, & Zhang, 2025)

8. Macro-architectural modalities

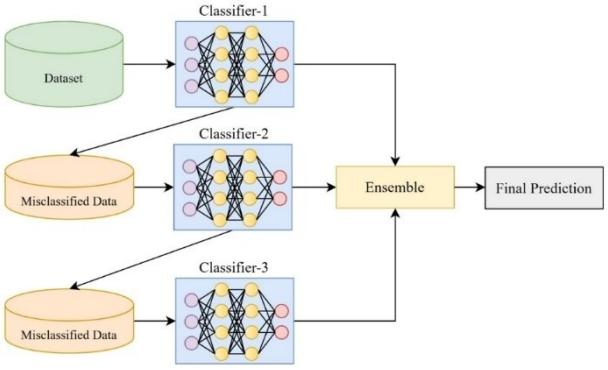
8.1. Bagging

In the bagging technique, we consider a series of models, called base learners. Through the bootstrapping technique, we assign to each of this models, a subset of the training data with replacement. The models are trained independently in parallel and, thanks to the diversity among them, we're reducing the risk of variance and overfitting. The outputs of these models are then finally averaged and this will be the last prediction.

(Breiman, 1996)

8.2. Boosting

In the boosting technique, the various models are trained sequentially. Each model focus on correcting the errors made by the preceding model by assigning higher weights to misclassified data points during training and this process is repeated iteratively. This method allows to reduce the bias. The final prediction comes from combining predictions of all individual models using a weighted voting method.



(Freund & Schapire, 1996)

8.3. Stacking

In stacking, we start by training different models on the same full training data. The models can also be of completely different kinds. After this first phase, a meta-model is trained based on the previous outputs: this model learns how to combine and what weight to assign to each model and is the one that will produce the final prediction.

(Wolpert, 1992)

8.4. Mixture of experts

The Mixture of Experts (MoE) is another kind of ensemble model, designed to improve the efficiency by using some "experts" to handle different parts of an input. We have three main components:

- Experts: Models specialized in some specific tasks
- Gating network: A router that chooses which experts to use
- Sparse activation: A method that makes sure only a part of the experts is activated for each input.

9. Training

9.1. Distillation

Architecture

In Distillation, we have two models: a pre-trained complex model, called the teacher and a yet-to-train simple and smaller model called the student. Both the teacher and the student are fed the same data and the goal is to train the student so to have similar a performance of the teacher.

Training

During training some unlabelled data is fed to the teacher that assigns a label and the student mimics its behaviour. In the Knowledge Distillation the student receives both the hard targets (ground truth labels) and the soft targets (the teacher's output probabilities, so not a

single result). Another distillation technique is using the teacher to generate more data. We can also give in input to the student the intermediate layers of the teacher model or we can use multiple teachers to aggregate their knowledge.

Advantages and Limitations

The advantages of distillation are:

- We can have a smaller model whose performance is closer to a complex one.
- Having a smaller model, inference is faster.
- Computational costs are reduced.

The disadvantages of distillation are:

- The smaller model only learns from the teacher and might inherit its shortcomings.
- In order to have an effective training we require a considerable amount of unlabelled data.

(Hinton, Vinyals, & Dean, 2015)

9.2. Reinforcement learning from human feedback

Architecture

In Reinforcement Learning from Human Feedback (RLHF), we use a pretrained LLM and we take multiple outputs from a given prompt. A human will then rate the outputs based on their quality and accuracy and deciding whether they're ethical or not (according to some categorical questions) and then a reward model is based on these answers. The reward model will assign a score to each answer and the initial model is fine-tuned so that it'll try to maximize the reward received.

Advantages and Limitations

The advantages of RLHF are:

- The model's outputs will be closer to what's acceptable for a human user.
- The feedbacks given by humans can improve the user experience with the model.

The limitations of RLHF are:

- The model is subject to human error, so if a reviewer gives a wrong feedback, the model won't know.
- The results of the model depend on the type of prompt that was given.
- The approach has a low scalability, because a higher number of prompts to evaluate, means that the reviewers will spend more time to give their feedbacks.

(Ouyang, Wu, & Jiang, 2022)

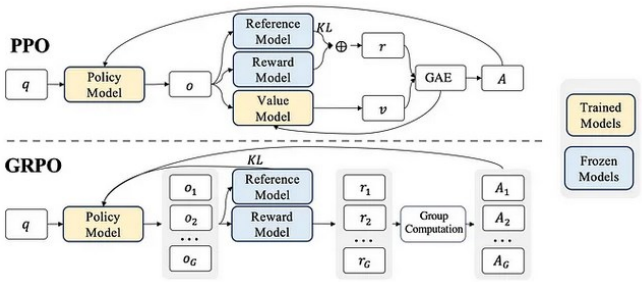
9.3. Reinforcement learning from AI feedback

In this case, another model is trained based on human feedback to understand the principles to make the judgments. This model will be then used to evaluate the pretrained model. In this way, we have a more scalable and less subjective RL technique.

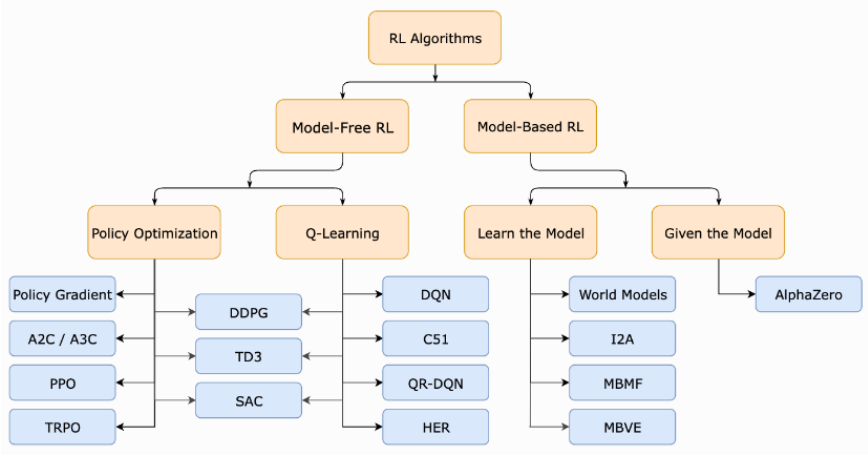
(Bai, Kadavath, & Kundu, 2022)

9.4. Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) is a reinforcement learning algorithm designed to solve the issue of having an overhead on a single critic model and improves the scalability. The model generates a group of responses from the same prompt and then the reward model assigns a score to each response based on how good each response is relative to the others in the group. After this, we compute the average of these answers, obtaining a score for the entire group, that will then be used to update the LLM's policy.



(Shao, Wang, & Zhu, 2024)



9.5. Parameter Efficient Fine-Tuning

The Parameter Efficient Fine-Tuning (PEFT), helps pretrained models to specialize to a specific task. There're two approaches:

Additive fine tuning

We add new parameters to the model.

- Adapter-based fine-tuning: we insert small adapter layers within the Transformer blocks so that we can have a better performance with only few more parameters. An additive adapter is:

$$Adapter\ x\ = W_{up}\sigma\ W_{down}x\ + x$$

Where, W_{up} is the up-projection matrix and W_{down} the down-projection matrix.

- Soft prompt-based fine-tuning: we add some adjustable vectors at the start of the input sequence. For example, in Prefix-tuning, we add these vectors to the keys and values and after the fine tuning, we use only the prefix vectors for the inference.

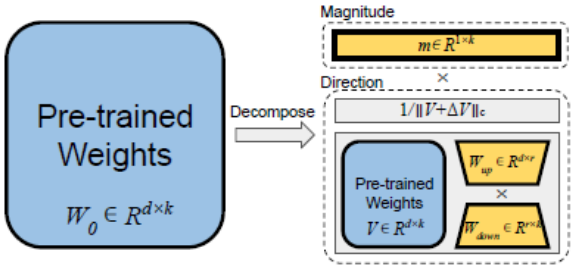
Selective PEFT

We perform a fine tuning of a subset of existing parameters.

- Unstructural masking: we learn a mask to the model weights or select a subset of the network or approximate the original problem.
- Structural masking: we organize the parameter masking in regular patters and then rank them or eliminate them or do a fine tuning only in some of them.

Reparametrized PEFT

We convert a model's architecture to another by changing its parameters. We can convert the weights or compress them for efficiency. For example, in DoRa, we decompose the pre-trained weights into two components, magnitude and direction and then we compress the directional part.



Hybrid PEFT

We can combine these techniques, for example, UniPELT integrates the Reparametrized PELFT with prefix-tuning and adapters.

(Han, Gao, & Liu, 2024)

10. Prompting

The following methods are ways to fine-tune a model in order to respond better to certain types of prompts

10.1.Zero-shot prompting

Through the use of instruction tuning (fine-tuning a pretrained language model on a mixture of tasks phrased as instructions), it's possible to improve the zero-shot prompting abilities of the model. Zero-shot prompting means that the prompt used to interact with the model won't contain examples or demonstrations; the LLM understands the task without any further explanation.

```
Input: Classify the text into neutral, negative or positive: "I think the vacation is okay"
Output: Neutral
```

(Wei, Bosma, & Zhao, Finetuned Language Models are Zero-Shot learners, 2022)

10.2.Few-shot prompting

Training a model for a zero-shot prompting could not be enough in case of more complex tasks. In the few-shot prompting, we provide to the model some examples that will help it to generate a response.

```
Input: Classify the text into neutral, negative or positive:
“This is awesome! - Positive”
“This is bad! - Negative”
“What a horrible day! - ”
Output: Negative
```

10.3.Chain of thoughts

In Chain of Thoughts (CoT), the model breaks down the prompt, so that the model can have a better understanding of the problem and this is effective especially with complex tasks, that require a guide to be solved. There’re many variants:

- Automatic Chain-of-Thought: we use a model to find patterns in pre-provided examples and use those patters on new problems.
- Zero-shot Chain-of-Thought: In Zero-shot CoT, we use pre-defined prompts.

```
Input: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3
tennis balls. How many tennis balls does he have now?
Output: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.
5 + 6 = 11. The answer is 11
```

(Wei, Wang, & Schuurmans, Chain-of-Thought Prompting Elicits Reasoning, 2023)

10.4.Meta prompting

In Meta prompting the focus is on the syntax and the structure rather than the specific content. This can be done by focussing on abstract examples rather than specific ones and by emphasizing a categorical and logical approach in analysing the prompt. This reduces the number of tokens and enhances the problem-solving abilities.

```
Problem statement:
    - Problem: [question to be answered]
Solution structure:
    1. Begin the response with “Let’s think step by step.”
    2. Break down the solution process clearly and logically and follow the steps.
    3. End the solution with the final answer encapsulated in LaTeX-formatted box.
    4. Finally state “The answer is [final answer to the problem]”
```

(Zhang, Yang, & Chi-Chih Yao, 2023)

10.5.Generated Knowledge prompting

In the Generated Knowledge prompting, the approach is to add a phase of knowledge generation to improve the answer. Given a prompt, a language model first uses the few-shot

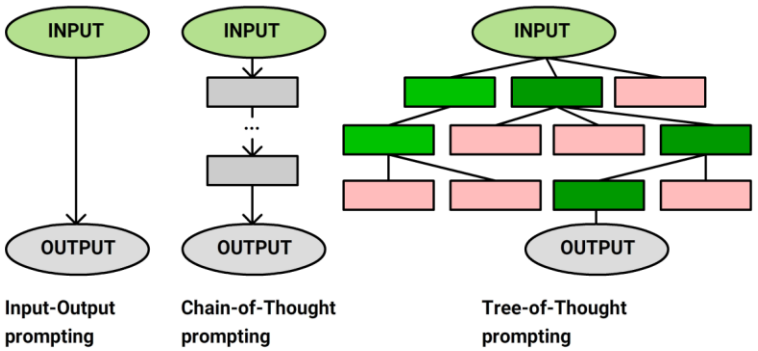
method to generate a sample of general knowledge related to the prompt, and then a second language model uses each of these knowledges to generate a possible answer.

```
Input: Is Greece larger than Mexico?
Knowledge: Greece is approximately 131,957 sq km, while Mexico is approximately
1,964,375 sq km, making Mexico 1,389% larger than Greece.
Output: Yes, Greece is larger than Mexico?
```

(Liu, Liu, & Lu, 2022)

10.6.Tree of Thoughts

The Tree of Thoughts (ToT) approach is an extension of the Chain of thoughts; indeed, it maintains a tree of thoughts, where each branch is a chain of thoughts. At each step only the best branches are kept and the model can continue its evaluation from those steps. This allows also to use searching techniques on trees such as breadth-first search or backtracking.



(Long, 2023)

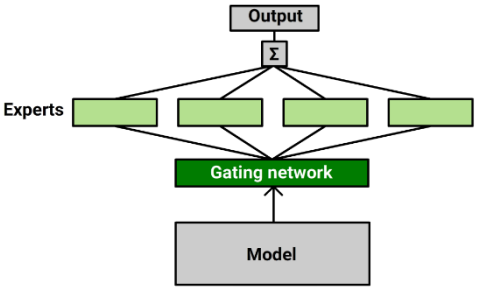
11. Mixture of experts

Now that we have analysed all the steps and components of an LLM, we can proceed with the specific study of the Mixture of Experts.

Architecture

The Mixture of Experts (MoE) is a kind of ensemble model, designed to improve the efficiency by using some "experts" to handle different parts of an input. We have three main components:

- Experts: Models specialized in some specific tasks.
- Gating network: A router that chooses which experts to use.
- Sparse activation: A method that makes sure only a part of the experts is activated for each input.



If used in a transformer architecture, the gating network is located after the self-attention layer.

Experts

An expert is an individual sub-network trained on a specific task. It can be a simple feed forward network or it can be a more complex component such as another MoE.

After training on various datasets, it has been observed that the experts specialize on groups of similar words (for example punctuation or verbs) rather than on languages.

Gating network and Sparse activation

The concept at the base of MoE is sparsity: in a dense model all the parameters are used for all the inputs, while in a sparse model, with conditional computation, some specific parts are activated/deactivated. This is why most of the parameters in a MoE are actually divided between the experts, while just a small part is in common and it's activated with every input. Some of these parameters are assigned to the gating network or router. A gating network is another layer of the model that decides to which expert it should route the input:

$$G_\sigma\ x\ = SoftMax(x \cdot W_g)$$

The gating network takes in input the entire sequence and then assigns to each token as many probabilities as experts (it's an $n \times m$ matrix, where n is the number of tokens and m is the number of experts). This means that the probabilities change based on the context.

In the Hard MoE, the output is given by the result of a single expert. Each input has a single expert assigned.

In the Soft MoE, the output y is instead given by a weighted sum of all the experts:

$$y = \sum_{i=1}^{n} G_i\ x\ E\ x$$

Where E_i is an expert and G is the weight given to that expert; if $G_i\ x\ = 0$, it means that we don't need to compute the operations of expert i.

The router strategy helps preventing problems like the ones related to the batch size: with large batches, we could overuse a specific expert by assigning it many tokens and leave the others untrained.

To prevent this, we can adopt the Sparse MoE (we activate a subset of experts) with noisy top-k gating, in which we're selecting the top-k experts with the highest affinity scores and sends the input data to them. By choosing more than one expert at a time, we'll train the gating network to route between experts and the noise is added to help load balancing: it's indeed possible that the gating network will always choose the first k experts, but with some additional noise, we encourage the model to give to all the experts the same importance.

Another important feature of the MoE is the Expert capacity, the number of tokens that can be processed by one expert:

$$Expert\ capacity = \left(\frac{tokens\ per\ batch}{number\ of\ experts}\right) \cdot capacity\ factor$$

Setting a capacity factor to a number greater than one means adding a buffer to the experts, in the case in which the token distribution is not uniform and an expert receives more tokens than another. When the buffer is full and that expert receives another token, that token will be either forwarded to the next layer via a residual gate or dropped completely. The higher is the capacity factor and the higher is the quality, but the communication costs will increase too.

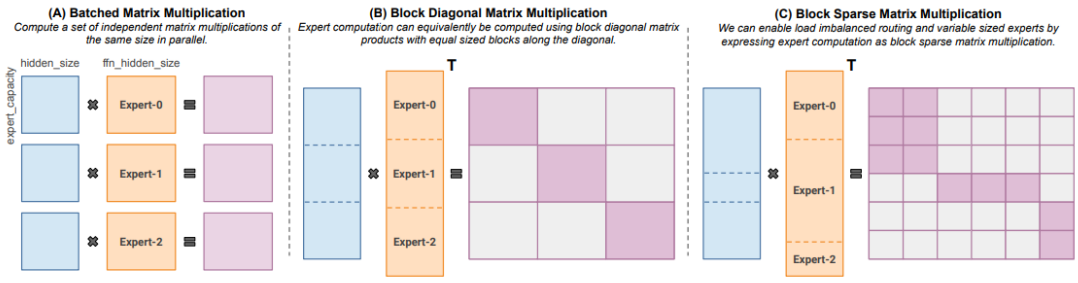
A variant of the Sparse MoE is the Adaptive MoE, in which the gating router learns also the number of experts to activate, so to have a variable number of experts per input, that improves the training phase while keeping a low number of parameters thanks to the sparsity.

Another routing technique is the Expert choice routing. Instead of letting the token decide the top-k experts, we let the expert decide the top-k token, so that each expert can decide which data they can handle best and we don't have the problem of untrained experts, but we could have the problem of having tokens not picked by any experts.

A variant of the entire structure of the Moe is the Hierarchical MoE, in which we have a tree of layers: the leaves are the experts and the nodes are the routers. In this type of model, we can have more fine-tuned decisions with an increase of the computational cost. For example, a 2-level hierarchical MoE, with a first order gating function $G_i(x)$, a second order gating function $G_{j|i}\ x$ and an expert $E_{j|i}$ the prediction will be:

$$y = \sum_{i=1}^{n} G_i\ x\ \sum_{j=1}^{n} G_{j|i}\ x\ E_{j|i}\ x$$

Traditionally all the experts have the same shape and are assigned the same number of tokens (so the computations can be expressed in terms of block diagonal matrix multiplication with identically sized blocks. The MegaBlocks architecture abandons these constraints and constructs a block diagonal matrix with variable sized blocks made up of many smaller blocks. This matrix can be efficiently computed using block-sparse matrix multiplication. Thanks to the block sparsity, the problem of having the dropout in tokens is solved.



Training

During the training phase, we need to train both experts and gating network. To train the experts, we need to consider them as an individual neural network. We divide the dataset in subsets, assign to each expert a subset and proceed with the usual training of a NN (so reducing the loss). To train the gating network, we provide the whole data fed into a model and, through a supervised learning, it should give as an output a probability distribution that allows to

choose the best expert. In the training phase, both experts and gating network are trained together and the loss function includes both components.

Having a sparse model rather than a dense one, means that the network is more prone to overfitting. To prevent this, we can use the dropout regularization (that makes however decrease the quality), both withing the experts themselves and a higher one for the sparse layers.

Another way to fine-tune the model is to freeze the parameters of the MoE and train only the remaining ones. This is because, even if the parameters of the MoE are the majority, the others are the ones in common with all the experts, so they'll be used with every single token.

Advantages and Limitations

The advantages of the MoE architecture are:

- Enhanced scalability: Thanks to sparse activation, MoE models can have billions of parameters without requiring too much computational power.
- Increased flexibility: new experts can be added to the model, without having to train again the entire system.
- Efficiency: Thanks to the routing, MoE can handle diverse tasks more efficiently than traditional models, making it faster and more accurate, as experts can focus on what they do best.
- Parallel processing: Experts can work independently, which allows for efficient parallel processing. This approach can cause faster training and inference times.

The limitations of the MoE architecture are:

- Training complexity: Training MoE models is a complex task and may lead to overfitting, so to assign too high weights to some models, ending up using only specific experts and discarding the others.
- Communication costs: In MoE models the communication between each component requires coordination and synchronization of the outputs from various experts and may lead to overhead.
- Expert Capacity: To ensure a balanced workload only a limited number of experts is chosen and also the number of inputs each expert can process simultaneously is limited. This strategy also assigns one expert per core, optimizing performance and resource management.
- Interpretability: Understanding the decision of a MoE model is even harder with respect to classical NLP models, because we have to understand each model singularly and how the gating mechanism is working.

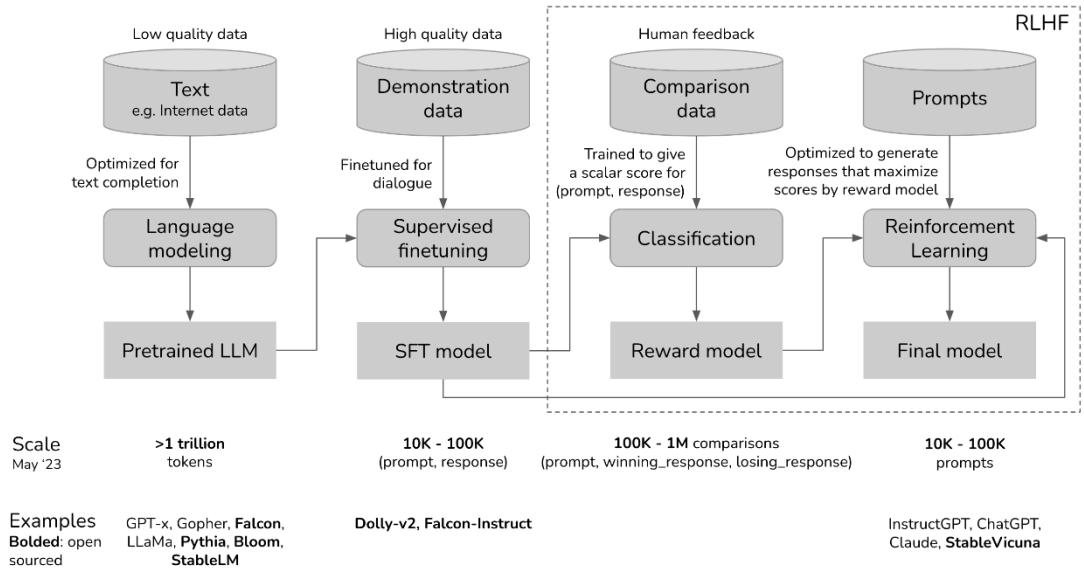
(Shazeer, Mirhoseini, & Maziarz, Outrageously Large Neural Networks, 2017)

(Zhou, Lei, & Liu, 2022)

(Gale, Narayanan, & Young, 2022)

12. Models

The general pipeline to train a model is this one:



The structure of the main LLM models is:

Model	Year	Tokenizer	Positional Encoding	Activation Functions	Model components
BERT	10-18	WordPiece	Learned PE	GeLU	Encoder-only transformer
GPT2	02-19	BPE	Absolute PE	GeLU	Transformer
PaLM	04-22	SentencePiece	RoPE	SwiGLU	Decoder-only transformer
DeepSeek LLM	11-23	BBPE	RoPE	SwiGLU	Transformer with GQA
LLaMA-3	04-24	BPE	RoPE	SwiGLU	Autoregressive decoder-only Transformer
Phi-3	06-24	BPE	RoPE	GeGLU	Decoder-only transformer
Gemma	07-24	BBPE	RoPE	GeGLU	Decoder-only transformer with GQA
Mistral-7B	07-24	BPE	RoPE	SiLU	Sliding window attention with GQA
DeepSeek V3	12-24	BBPE	RoPE	SwiGLU	Transformer with MLA

And the structure of the main models using MoE is:

Model	Year	N experts	Activated experts	Total parameters	Activated parameters
GPT 4*	03-23	16/8	???	???	110B/220B
Mixtral 8x7b	12-23	8	2	46.7B	12.9B
DBRX	03-24	16	4	132B	36B
DeepSeek V3	12-24	256	8 (1 shared)	671B	37B
Llama 4 Scout	04-25	16	???	109B	17B
Llama 4 Maverick	04-25	128	???	402B	17B

* There's no official technical report about the model, so these are just hypotheses

13. DeepSeek R1

Released on the 20th of January 2025, DeepSeek R1 has the same architecture of DeepSeek V3, but has also reasoning abilities.

For the pre-training phase the strategies adopted were the deduplication, filtering and remixing ones, while for the tokenizer BBPE was chosen.

The architecture of this model is based on the Transformer one, with important changes.

During the inference phase, the transformer uses as context all the past tokens, but this is inefficient, because it requires to load again past tokens that have already been processed before, meaning that it's evaluating again all the previous keys and values. This problem is solved by caching these pairs in a KV cache. With long sentences however, this cache requires too much space and this is why we pass first to the Grouped Query Attention (GQA) and then, since it compromises model's quality, to Multi-head Latent Attention (MLA). MLA should compromise model's quality too, but it's more effective than GQA because it's able to capture better the similarities between the vectors.

Another important change is the usage of mixture-of-experts (MoE). However, for the risk of routing collapse (training too much an expert and forgetting the others), balanced routing was adopted. This model however implies that all the experts should be used with a uniform probability distribution, while in reality some models will be used more frequently than the others (for example the model for the English language, will be used more than a model about a specific task). In DeepSeek this problem was solved by assigning to each model a probability p of being used that in some models is 1.

About the inference step, DeepSeek predicts one token and then uses that token again to predict the next one, so at each step we have a multi-token prediction. This additional token can be used in the next step to see whether it should accept it or not and to get additional information about its own decoding. DeepSeek R1 uses multi-token prediction up to the second next token.

The most recent LLMs use RL to improve, for example ChatGPT RLHF. In DeepSeek R1, instead of having an actor network that generates the output and a critic network that estimates the quality, we have the Group Relative Policy Optimization. In addition to this, DeepSeek-R1 uses the prompting technique of Chain of Thought, with an additional cold-start phase with high quality data to prevent language mixing.

14. Practical experimentation

14.1. The data used

In order to test a model using MoE I first needed to find a dataset. I decided to choose it from a competition available on Kaggle. I chose the one that was still open for the submission and the most interesting one. The aim of the competition is indeed to solve a still open problem, which is predicting the shape of an RNA molecule given the sequence of its nucleotides.

The RNA molecule 1A60 described by the sequence

GGGAGCUCAACUCUCCCCCCCUUUUCCGAGGGUCAUCGGAACCA



Two datasets were provided for training. The first one contains the input and it's structured in this way:

- target_id: The identifier of the sequence; this id is the same as the entry in the Protein Data Bank.
- sequence: The RNA sequence and the actual input that we will provide to the model.
- temporal_cutoff: The date in yyyy-mm-dd format that the sequence was published.
- description: Details of the origins of the sequence.
- all_sequences: The sequences of all molecular chains present in the complete structure surrounding the original sequence.

The second dataset contains the target, so the coordinates of each nucleotide. The dataset is structured in this way:

- ID: that identifies the target_id and residue number, separated by "_".
- resname: The RNA nucleotide (A, C, G, or U) for the residue.
- resid: residue number.
- x_1,y_1,z_1,x_2,y_2,z_2,...: Coordinates (in Ångströms) of each nucleotide. In some targets, the experimental method has captured more than one conformation and these coordinates are written in the other columns.

The input dataset contains 5119 sequences, whose length spans from 4 nucleotides (1Y1Y_P) to 1513 (1N34_A).



The RNA molecule 1N34_A

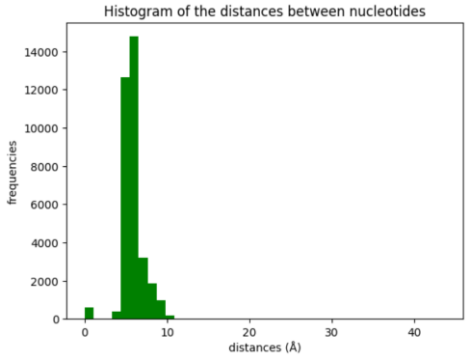


The RNA molecule 1Y1Y_P

Images taken from https://www.rcsb.org/

A fundamental data that has to be considered in the training phase is the distance between each pair of nucleotides; this information evaluated on our dataset, shows that most of them fall within a range of 5-7 Ångström (Å) (0.5-0.7 nm).

The



14.2.Preprocessing

The preprocessing phase acted both in the sequences and in the labels datasets.

In the first one, not all the sequences were represented by 'A', 'C', 'G', 'U', but there were also some additional letters used as placeholder. For example, the RNA sequence 8XPP_3 was inserted as UGUUCGACGACUCUCUX

In the second one, some sequences were completely lacking of the coordinates and were therefore excluded from the train, since they were unusable.

14.3.Approaches to the problem

Since we're dealing with non-discrete data (the coordinates), it's not easy to process them with a language model, but we can still find multiple approaches:

- Use a pretrained LLM and formulate a prompt containing the input and ask for a structured output.
- Modify the output layer of a non-pretrained language model so that it has as many dimensions as coordinates instead of the canonical output (as big as the vocabulary).

- Tokenize the data and use a non-pretrained language model with these new defined tokens.

From the first to the last, the model is more and more customized for the specific task.

14.4.The model chosen

I decided to use the third approach, so I tokenized the data.

The model I used is the non-pretrained DeepSeek V2, with some adaptations for the specific task.

The parameters of the code for the original model of DeepSeek-V2 are:

Parameter	Value	Description
`max_batch_size (int)`	8	Maximum batch size. It's the number of token sequences that the model receives in input during training.
`max_seq_len (int)`	4096*4	Maximum sequence length of tokens.
`dtype (Literal["bf16", "fp8"])`	Bf16	Data type for computations. Bf16 has a wide range, because it has an 8-bit exponent and 7-bit mantissa. Fp8 has two variants; the one used in Pytorch (E4M3) has a 4-bit exponent and a 3-bit mantissa.
`vocab_size (int)`	102400	Vocabulary size; the total number of unique tokens.
`dim (int)`	2048	Model dimension. It's the input and output dimensionality.
`inter_dim (int)`	10944	Intermediate dimension (the hidden layer) for MLP layers.
`moe_inter_dim (int)`	1408	Intermediate dimension (hidden layer) for MoE layers.
`n_layers (int)`	27	Number of transformer layers. It's the number of encoders and decoders that are in the architecture
`n_dense_layers (int)`	1	Number of dense layers in the model. It's the number of feed-forward-networks before the MoE
`n_heads (int)`	16	Number of attention heads. It's the number of heads that are in each encoder/decoder.
`n_routed_experts (int)`	64	Number of routed experts for MoE layers.
`n_shared_experts (int)`	2	Number of shared experts for MoE layers.
`n_activated_experts (int)`	6	Number of activated experts in MoE layers (it's the top-k experts).
`n_expert_groups (int)`	1	Number of expert groups for routing in the MoE.

n_limited_groups (int)	1	Number of groups to route inputs to in the MoE.
score_func (Literal["softmax", "sigmoid"])	SoftMax	Scoring function for MoE routing.
route_scale (float)	1.	Scaling factor for routing scores.
q_lora_rank (int)	0	LoRA rank for query projections.
kv_lora_rank (int)	512	LoRA rank for key-value projections.
qk_nope_head_dim (int)	128	Dimension for query-key projections without positional embeddings.
qk_rope_head_dim (int)	64	Dimension for query-key projections with rotary embeddings.
v_head_dim (int)	128	Dimension for value projections.
original_seq_len (int)	4096	Original sequence length.
rope_theta (float)	10000.0	Base for rotary positional encoding used in θ or ω.
rope_factor (float)	40	Scaling factor for extended sequence lengths.
beta_fast (int)	32	Fast beta correction factor. It's the lower bound for the number of rotation.
beta_slow (int)	1	Slow beta correction factor. It's the upper bound for the number of rotations.
mscale (float)	1.	Scaling factor for extended attention.

For my problem I decided to use 256 dimensions for the embedded input and 2 blocks of layers (one dense and one MoE). The MoE has 7 experts with one activated expert per input.

In total, the model has 7.35 million parameters.

14.5.Training

Phases

DeepSeek released only the code for the inference phase, not the training one. For this reason, I had to do small but complex adaptations.

Instead of having the model learn the task directly, I decided to perform in advance two preliminary stages:

1. Learning about the input itself: Is the model working on an elementary task?
2. Learning about positions: Is the model capable of locate information inside the sequence?
3. Learning about spatial geometry: Is the model able to process and train on geometrical data?

Normalization and tokenization

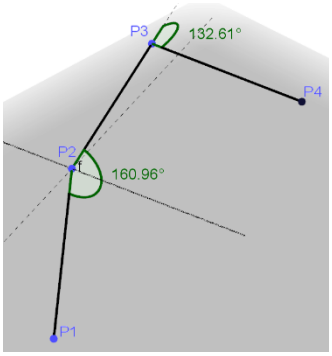
In order to tokenize the data, a small dictionary of tokens was used. There're base tokens and custom ones.

The base tokens are the ones of the nucleotide and they're one for each nucleotide (A, C, G, U).

The custom tokens are the ones for the positions, so one token for each position of the nucleotide in the sequence ($P1, P2, P3, \ldots$), the ones for distances and angles ($D1, D2, D3, \ldots, A1, A2, A3, \ldots$) and the special tokens for questions ($<SEQ>, </SEQ>, <NEXT_POS>, </NEXT_POS>, <WHO_AT>, </WHO_AT>, \ldots$)

Regarding distances and angles the tokenization happened in two steps:

1. The coordinates were converted to distances and angles. Given four consecutive points, we know that we can represent them by considering the distances between each consecutive pair and the angles between two consecutive segments. This method guarantees translational and rotational invariance.
2. We can finally convert the values in finite ranges and use a token per range: for example, the distances from 5.0 Å to 5.2 Å is token $D37$ and the angles from 10.1° to 10.3° are the token $A12$. This is regulated by a hyperparameter; the higher it is and the more precise is the conversion.



In this representation we're considering two angles: the angle between a triplet of points and the one between the plane of that triplet and the segment between the last point and its consecutive.

In order to finally convert the coordinates to distances and angles, we need to normalize the rotation and the starting position of the sequence: we translate the first point to 0, we align the first segment to the z-axis and the yz plane. In this way all the sequences will start at the origin and will proceed in the same direction.

Once we have normalized the sequences in this way, we start to read the sequence from the first point, whose coordinates will be (0, 0, 0). In order to discover the coordinates of the following point, we choose a direction based on the two angles and then we move as much as the distance between the first and second point tells us.

When the model will return a token, we need to detokenize the output. To do so, we will convert the angle and distance tokens to their corresponding ranges and we will choose as single number the average of each range. For example, if the token we get is $D37$, we will convert it to 5.1 Å.

Learning about the input itself

In this phase we verify that the model reads and interprets correctly the input that we provided.

The ‘repeat sequence’ question gives in input to the model the entire sequence and optionally a part of the sequence. If the part of the sequence is provided, the model has to continue from it until the end of the sequence; if it’s not provided, the model has to repeat the whole sequence:

$$< SEQ >, C, A, G, C, A, G, U, </SEQ >, < REPEAT_SEQ >$$

The input is the sequence CAGCAGU and the target is C (and then A, G and so on).

$$< SEQ >, C, A, G, C, A, G, U, </SEQ >, < REPEAT_SEQ >, C, A$$

The target is the nucleotide G (and then C, A and so on).

$$< SEQ >, C, A, G, C, A, G, U, </SEQ >, < REPEAT_SEQ >, C, A, G, C, A, G, U$$

The target is the closing token $</REPEAT_SEQ >$

Learning about positions

In this phase the model learns to answer simple questions about the position of the nucleotides in the sequence.

The ‘next position’ question asks to the model the following position token, given the whole sequence and a position token. If the position token is the last one in the sequence, it has to return the first one. In this way the model learns how to distinguish the position tokens from the others:

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, P5, A, P6, G, </SEQ >, < NEXT_POS >, P5, </NEXT_POS >$$

The target is $P6$.

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, P5, A, P6, G, </SEQ >, < NEXT_POS >, P7, </NEXT_POS >$$

The target is $P1$.

The ‘previous position’ question asks for the previous position. In:

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, P5, A, P6, G, </SEQ >, < NEXT_POS >, P5, </NEXT_POS >$$

The target is $P4$.

The ‘who at’ takes in input the sequence and, given a position, returns the nucleotide in that position:

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, P5, A, P6, G, P7, U, </SEQ >, < WHO_AT >, P4, </WHO_AT >$$

Should return C.

Learning about spatial geometry

In this phase the model receives in input the coordinates of the nucleotides too and learns to answer questions about them.

In the ‘distance’ question, given two points, returns the distance between them:

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, </SEQ >, < DISTANCE >, P1, P2 </DISTANCE >$$

Should return $D7$ (the tokenized version of the distance between the nucleotides in position $P1$-$P2$, that are C-A).

In the 'angle' question, given three points, returns the angle between them:

$$< SEQ >, P1, C, P2, A, P3, G, P4, C, P5, A, P6, G, </SEQ >, < ANGLE >, P1, P2, P3 </ANGLE >$$

Should return $A3$ (the tokenized version of the angle between the segments of the nucleotides in position $P1$-$P2$ and $P2$-$P3$, that are C-A, A-G).

Post-learning analysis

All the sources I read pointed out to extreme difficulties in learning MoE architecture especially regarding the router component and this is exactly what I encountered.

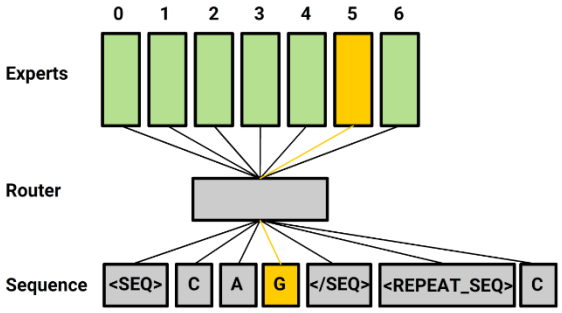
Indeed, in some tasks of the total 7 the training phase, the model went overfitting on the gate layer, meaning that it was ignoring all the experts but the same one. I read about successful techniques to overcome this problem, but they implied deeply modifying the base architecture, which was beyond the scope of this work.

In the tasks where the overfitting didn't occur, the model performed relatively well:

TASK	REPEAT_SEQ	NEXT_POS	PREV_POS	WHO_AT	DISTANCE	ANGLE1	ANGLE2
ACCURACY TEST	99%	95%	95%	83%	58%	53%	55%

In the tasks in which the model didn't overfit, we can analyse how the router network worked. When the model was asked to assign an expert to a sequence made only of token, it assigned all the position tokens to the expert 2 and 3, the nucleotides to the 5 and the question tokens to the others.

When the model was asked to assign an expert to a complete sequence of nucleotides and question tokens, the first were split between the experts 3 and 6 and the latter always to the expert 3.



All the tokens in this image are processed in parallel. Here I'm just highlighting the expert assigned to one token.

Accuracy of MoE vs Dense model with the same number of parameters

Criteria	MoE (single GPU)	Dense (single GPU)
Memory distribution	35 MB*	35 MB*
Inference time**	13 ms	10 ms
Speed of learning***	52 min on 8 epochs (average for each task)	1 h on 12 epochs (average for each task)

* Not considering the datasets but just the initialization of the architecture.

** Up to sequences of 30 nucleotides.

*** The training was stopped when accuracy was similar to the one presented in the previous slides on all tasks but the ones that went overfitting.

15. Conclusions

The results that I got from my experimentation confirm the ones already obtained by the researchers in this field. Here I present in a generalized way the results with single and multiple GPU systems too.

Criteria	Single GPU (what I confirmed with my experimentation)	Multiple GPUs
Memory distribution	MoE and dense model are equivalent	Each expert in the MoE can be assigned to a GPU. Much harder to do in the case of the dense model
Inference time	MoE is slightly slower, because it has the additional calculation for the gate layer	MoE is faster because we won't activate all the GPUs, but just the ones dedicated to the token.
Speed of learning	MoE is faster because each expert is a smaller MLP with respect to the equivalent layer in the dense model	
Scalability	Not applicable (depends on the specific GPU)	MoE is more prone to scalability (Google is deploying a 1 trillion parameters model that uses MoE)

This study has caught on my need of dealing with a wide-ranging deep learning technique. I found this model very interesting and with a bright future, so I believe it's inspiring to continue to study this topic in depth. I'm eager to experiment this model on its suitable environment that is a multi-GPU architecture.